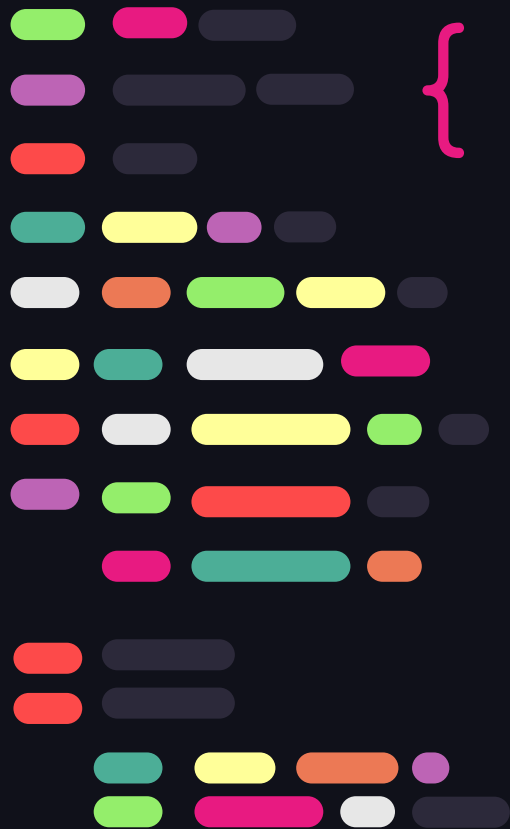




Revature Project 0

Bank of CLI

< By Miguel Fernandez >



Table of contents

01 Project Description

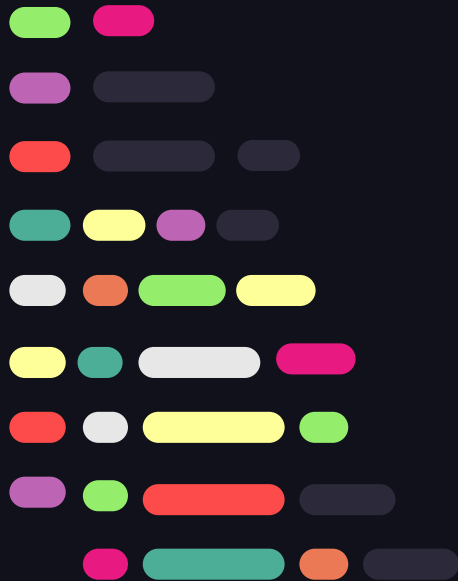
I will state the problem statement and requirements.

02 Design Documents

I will show and explain the documents created to design the application

03 Deposit Overview and Demo

You can describe the topic of the section here





01 { ..

Project Description



} ..



Project Description



The goal of the project was to build a functional banking application that runs entirely within the terminal.

MVP Features:

- **Secure Access:** Users must be able to register and login using a unique `Account ID` and `PIN`.
- **Balance Management:** Users can check their current account balance at any time.
- **The Transaction Engine:**
 - **Deposit:** Add funds to an account.
 - **Withdraw:** Remove funds (the system must prevent overdrawing!).
 - **Transfer:** Move money securely between two different accounts.
- **Audit Trail:** Users can view their recent transaction history.
- **System Logging:** The application must maintain a log file to track activity. You are required to use:
 - **INFO:** To record successful actions (e.g., "User successfully logged in").
 - **ERROR:** To record failures or security risks (e.g., "Incorrect PIN entered" or "Database connection lost").





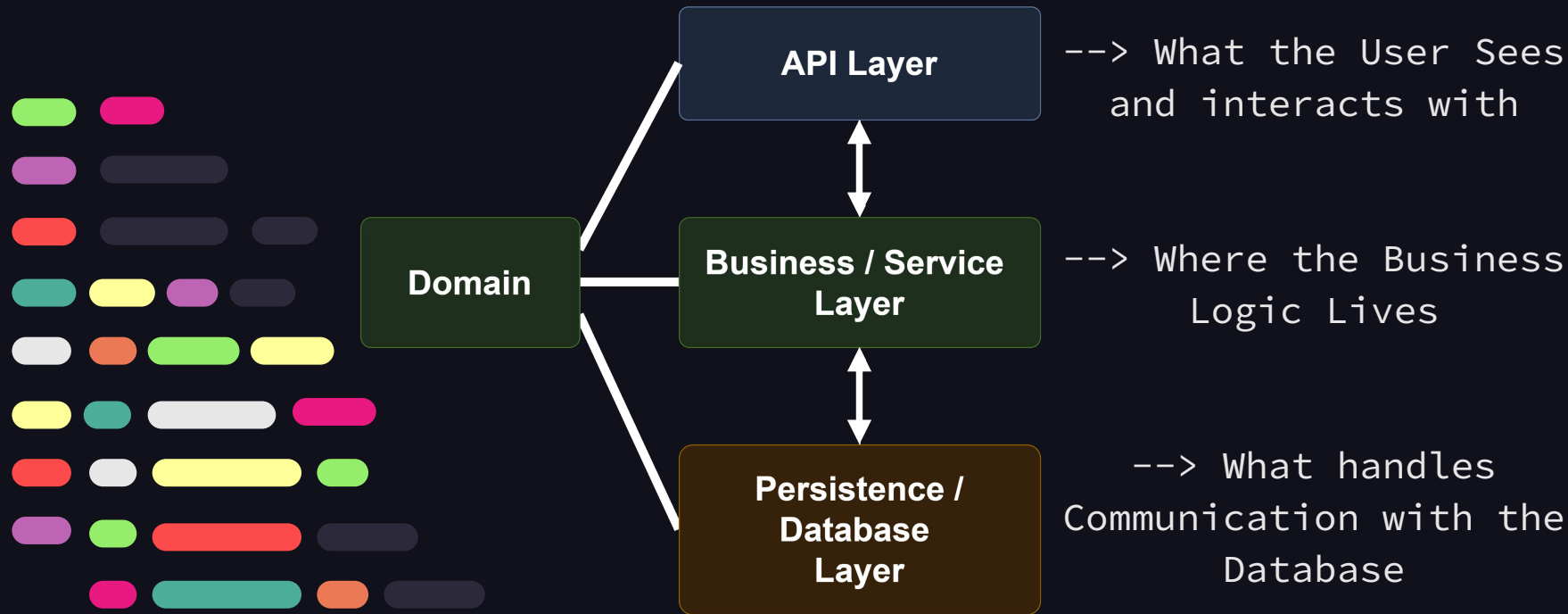
02 { ..

Design Documents

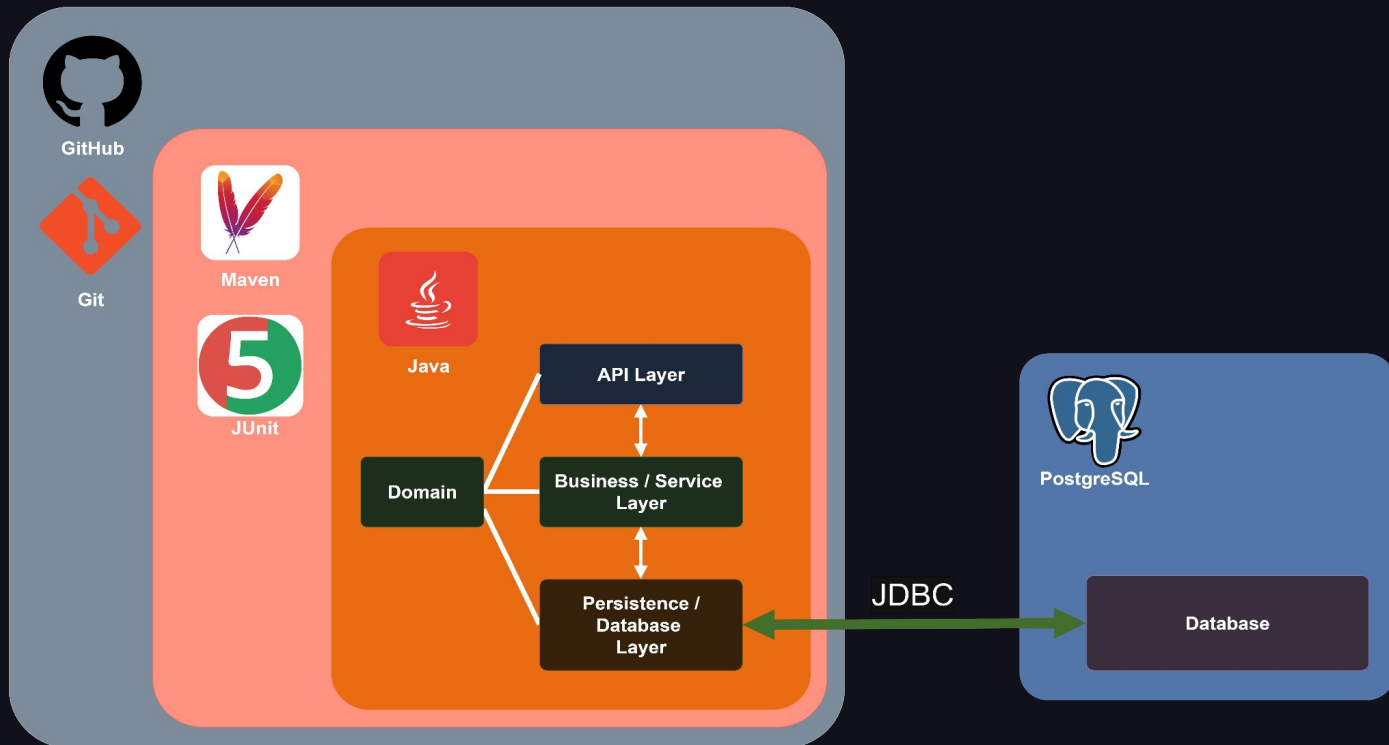


} ..

Architecture Design



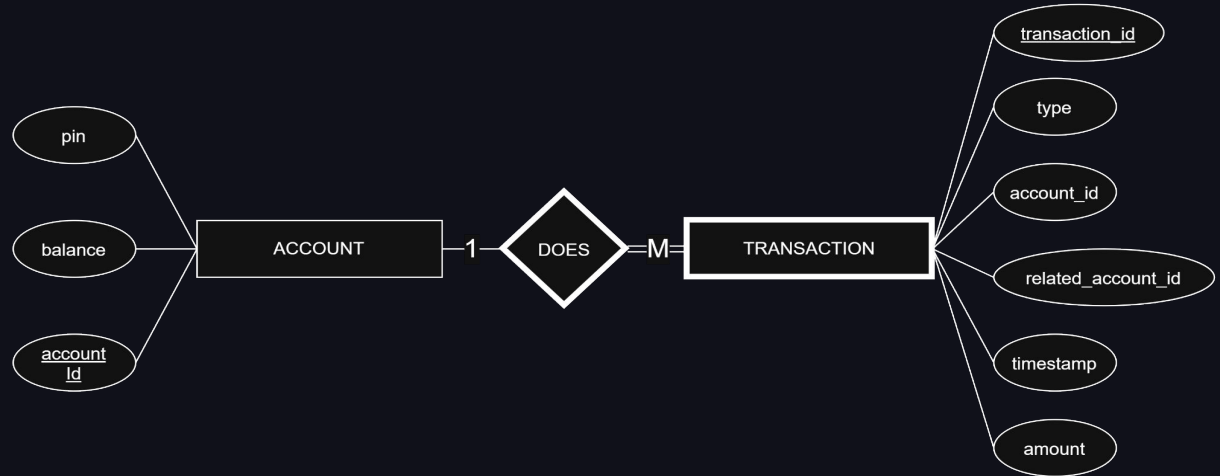
Tech Stack



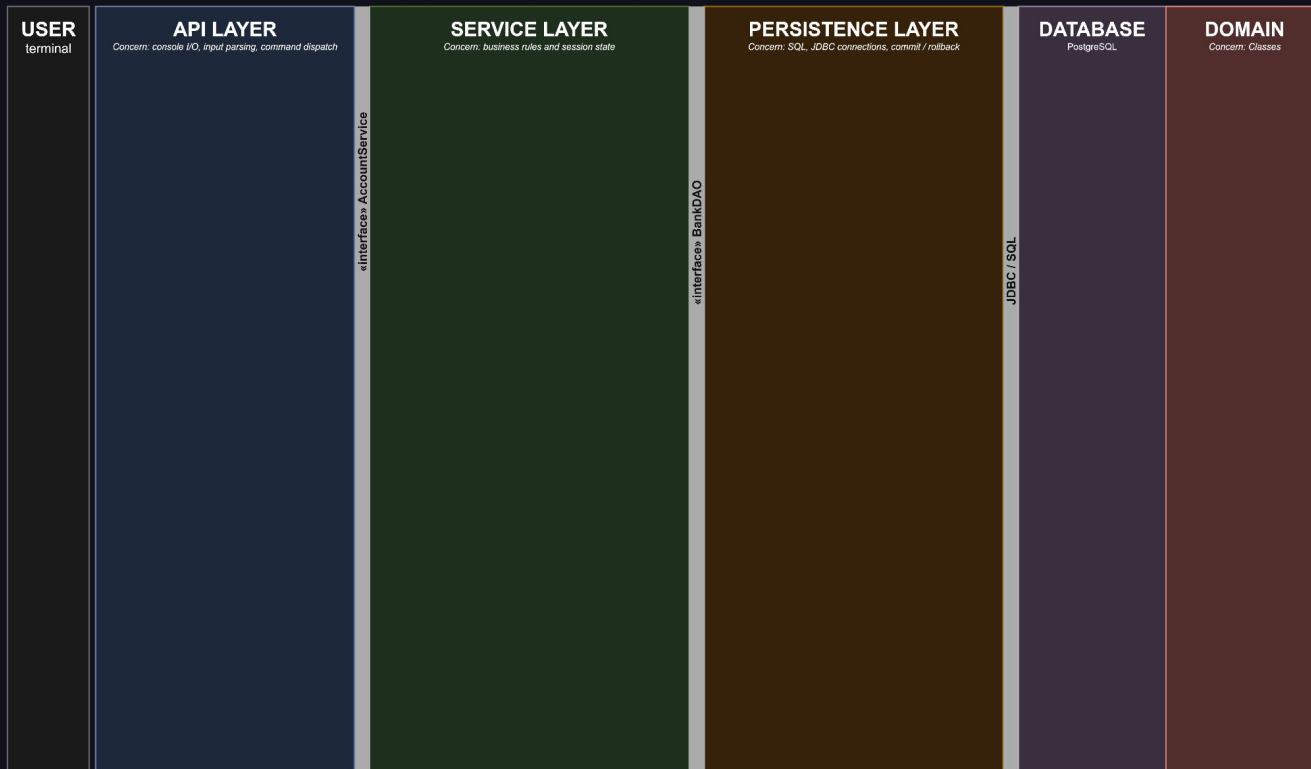
Database Schema



- **Secure Access:** Users must be able to register and login using a unique `Account ID` and `PIN`.
- **Balance Management:** Users can check their current account balance at any time.
- **Transactions:** User must be able to deposit, withdraw, and transfer money.
- **Audit Trail:** Users can view their recent transaction history.



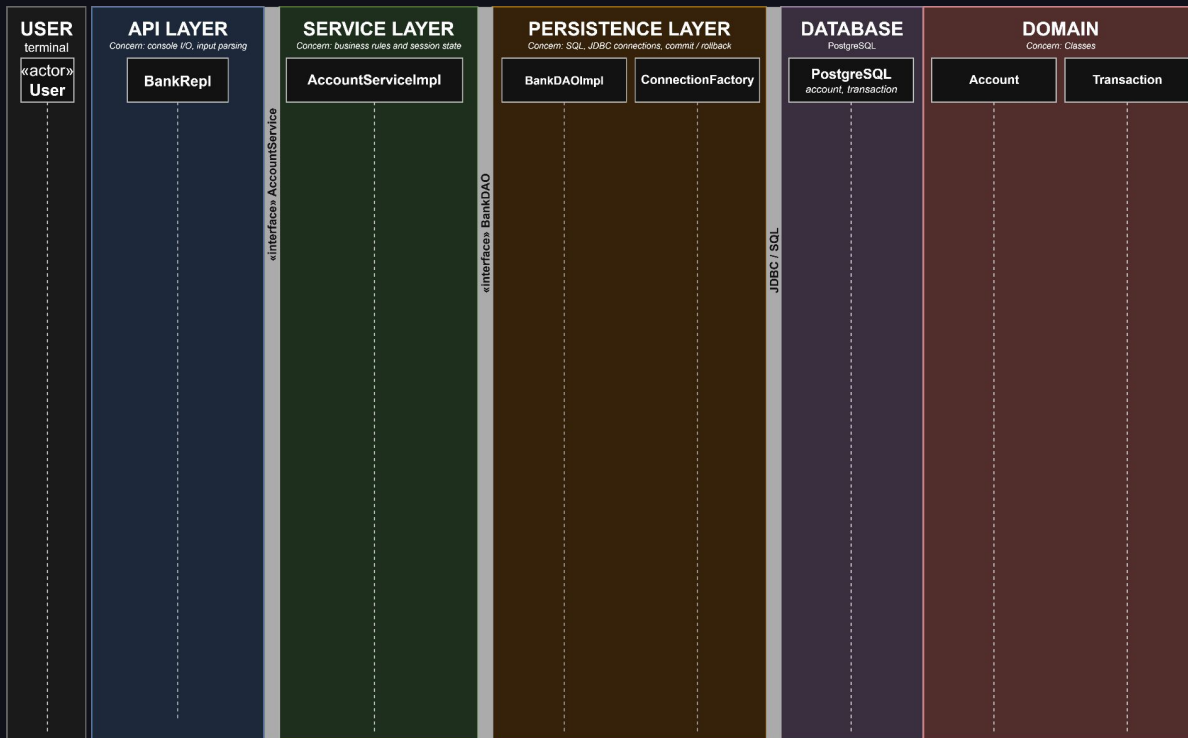
Project Structure



Project Structure



- ✓ api
 - BankRepl.java
 - Main.java
- ✓ domain
 - Account.java
 - Transaction.java
- ✓ persistence
 - BankDAO.java
 - BankDAOImpl.java
 - ConnectionFactory.java
- ✓ service
 - AccountService.java
 - AccountServiceImpl.java

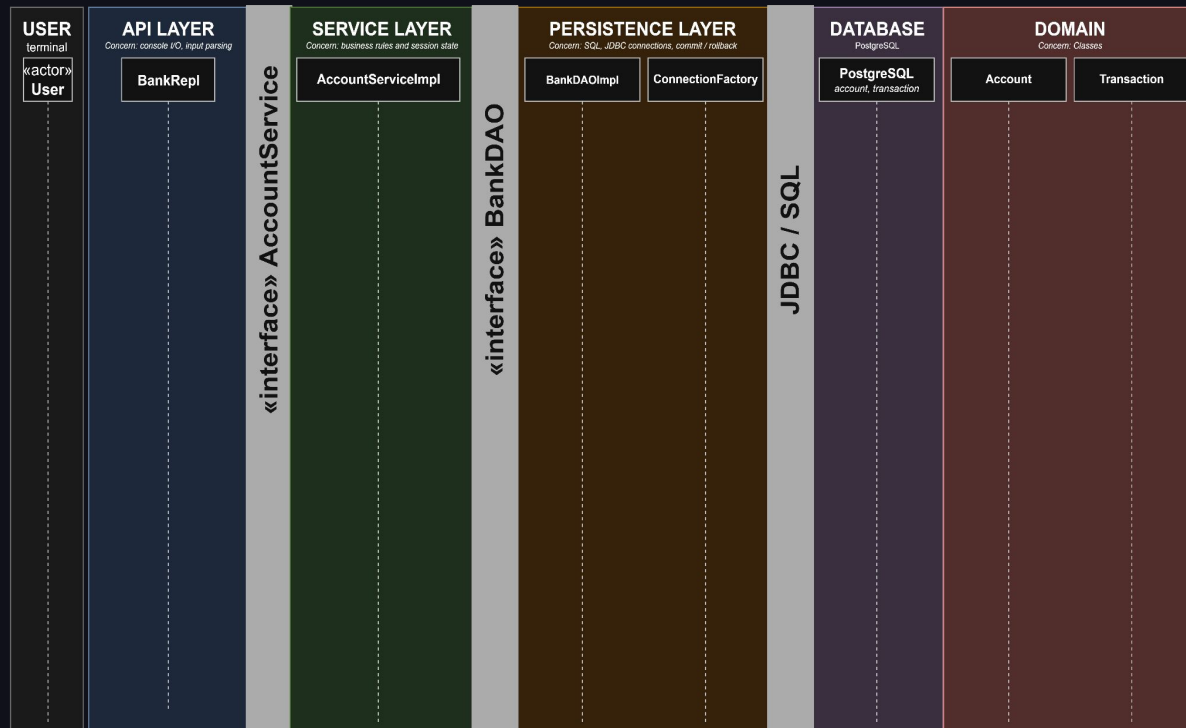


OOP Pillars:

Abstraction and Polymorphism



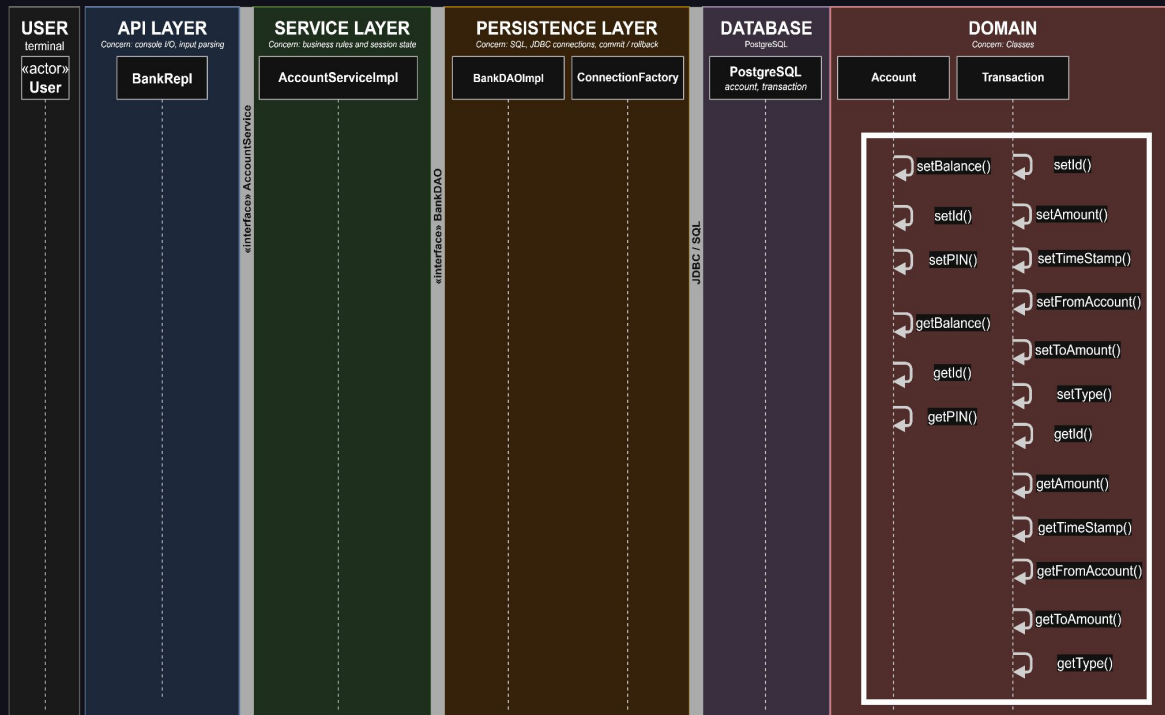
- ✓ api
 - BankRepl.java
 - Main.java
- ✓ domain
 - Account.java
 - Transaction.java
- ✓ persistence
 - BankDAO.java
 - BankDAOImpl.java
 - ConnectionFactory.java
- ✓ service
 - AccountService.java
 - AccountServiceImpl.java



OOP Pillars: Encapsulation and Inheritance



- ✓ api
 - BankRepl.java
 - Main.java
- ✓ domain
 - Account.java
 - Transaction.java
- ✓ persistence
 - BankDAO.java
 - BankDAOImpl.java
 - ConnectionFactory.java
- ✓ service
 - AccountService.java
 - AccountServiceImpl.java



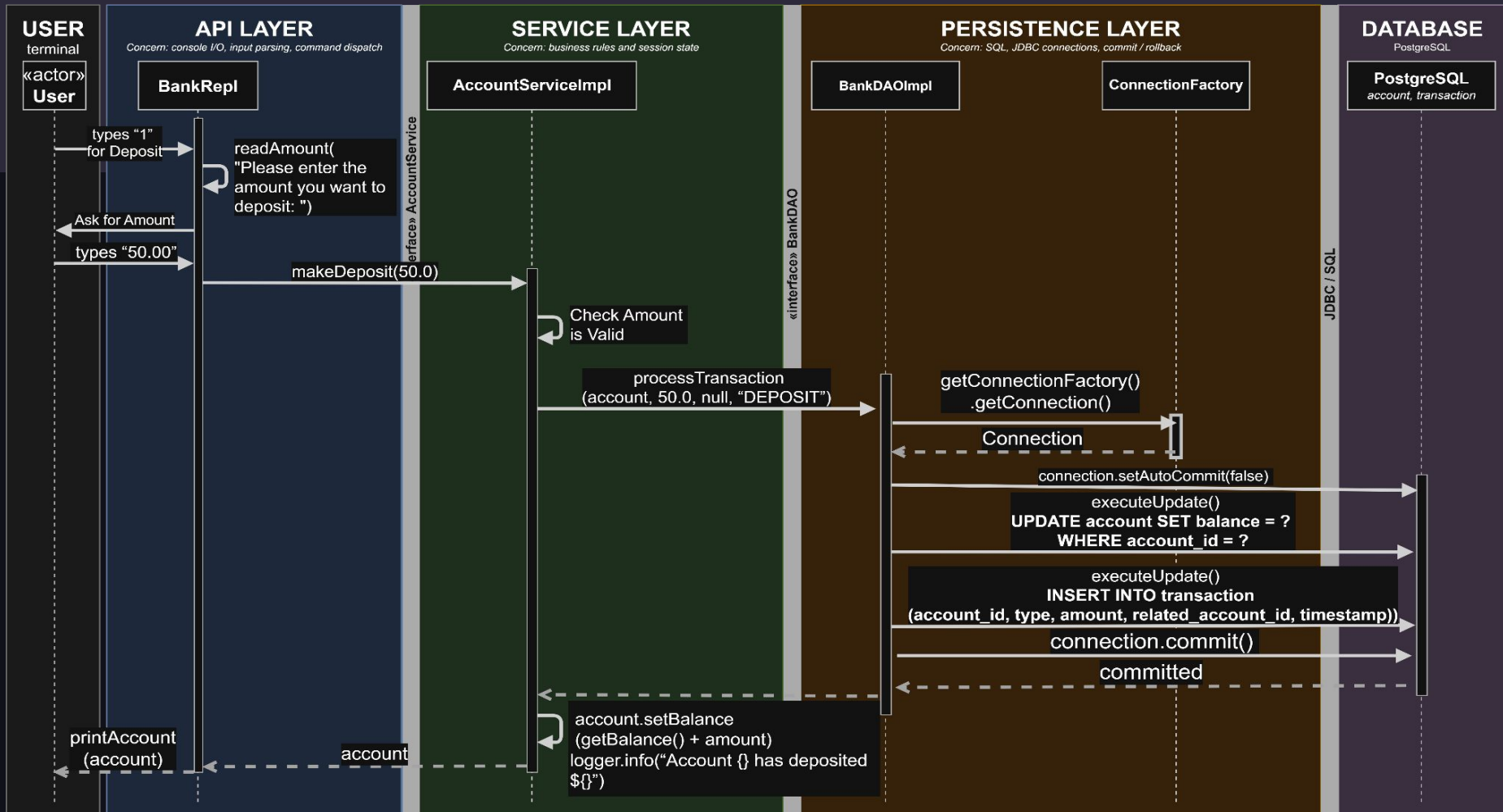


03 { ..

Deposit Overview



} ..



USER

terminal

«actor»
User

API LAYER

Concern: console I/O, input parsing, command dispatch

BankRepl

SERVICE LAYER

Concern: business rules and session state

AccountServiceImpl

PERSISTENCE LAYER

Concern: SQL, JDBC connections, commit / rollback

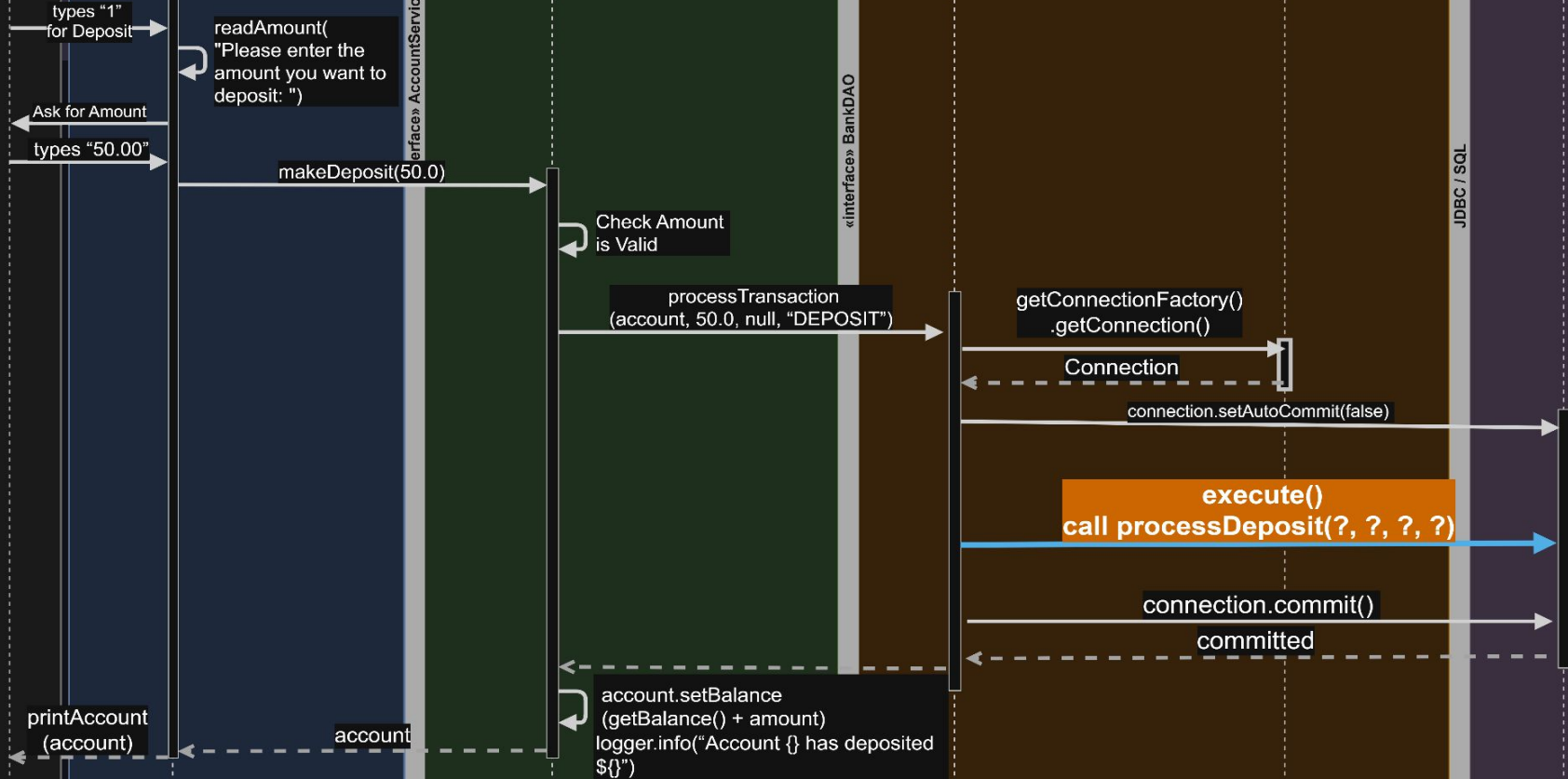
BankDAOImpl

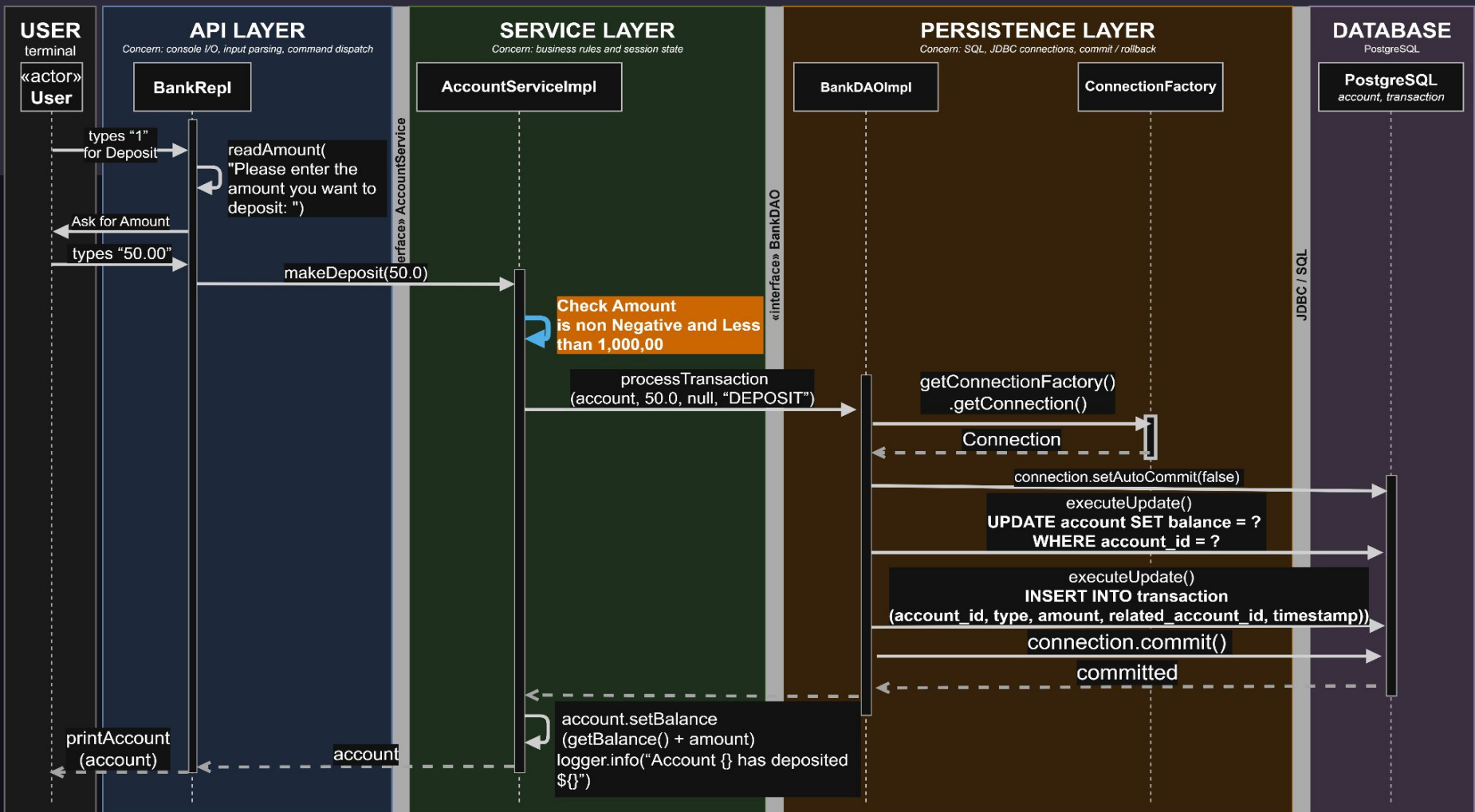
ConnectionFactory

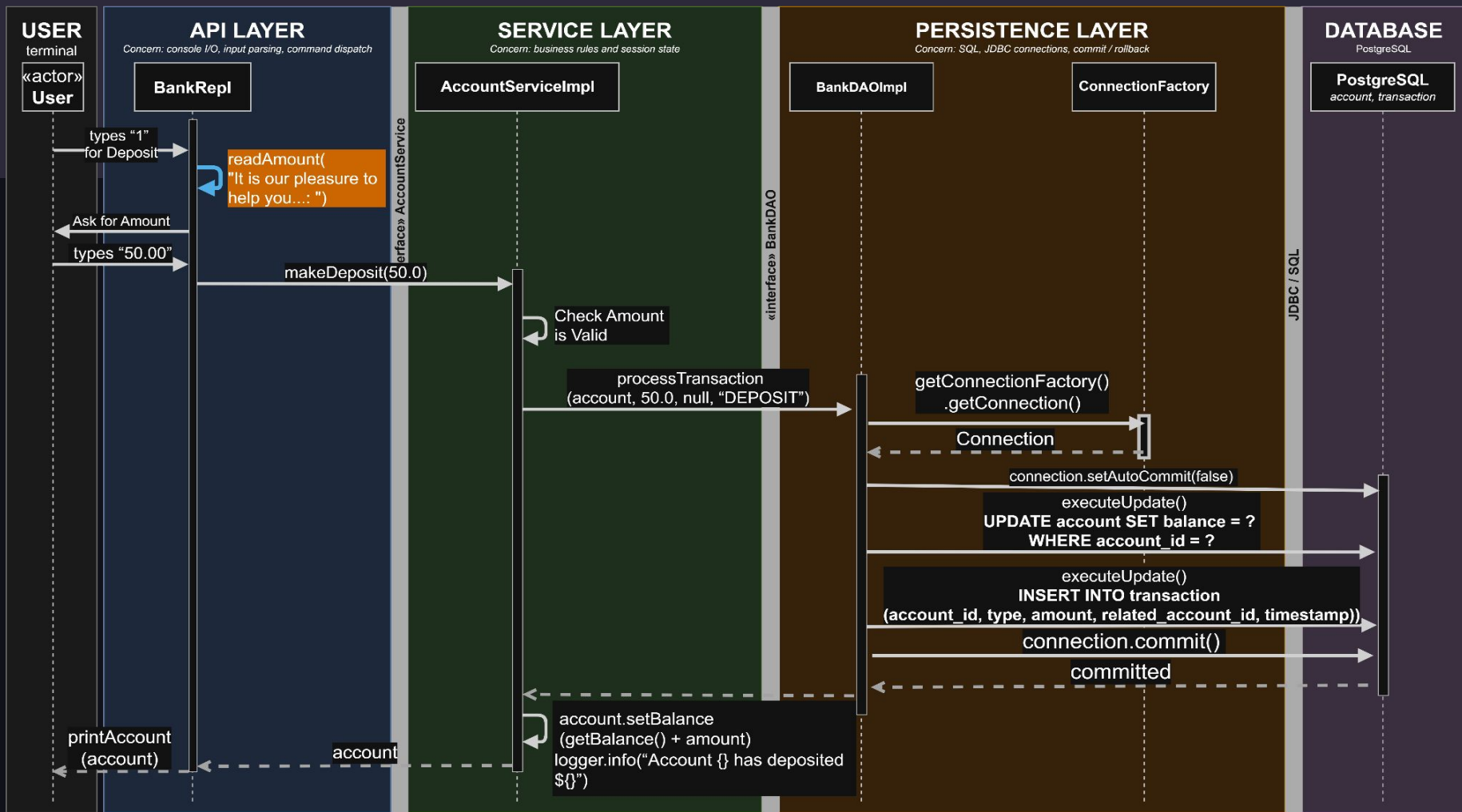
DATABASE

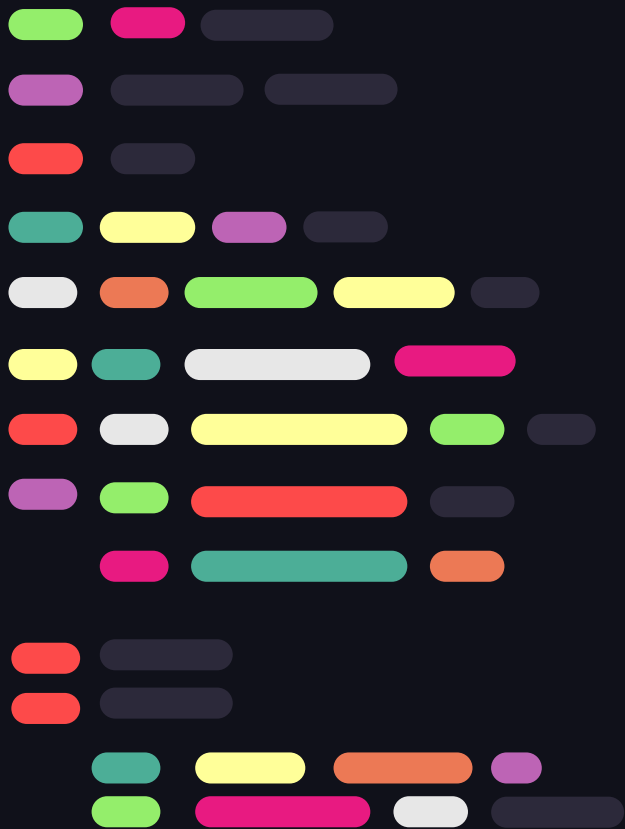
PostgreSQL

PostgreSQL
account, transaction



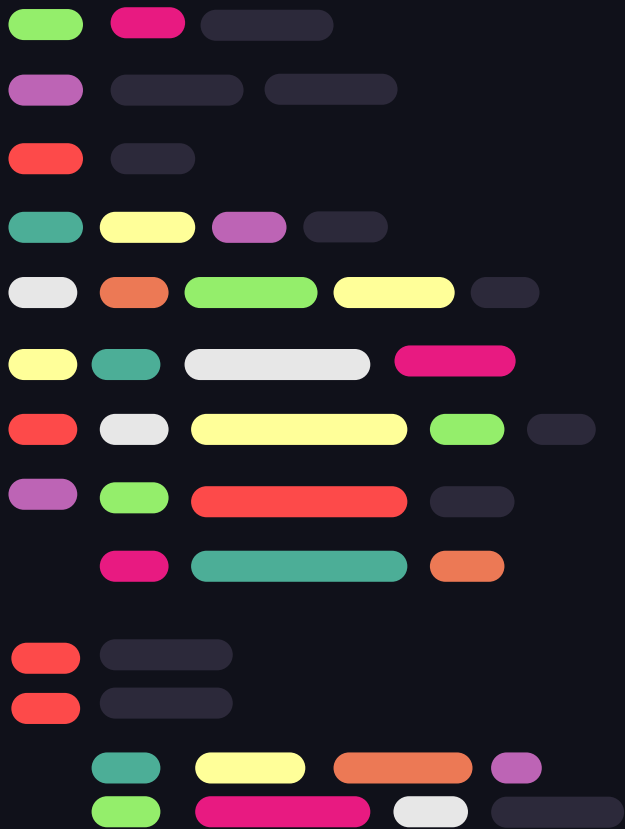






{ Demo ! }





{

..



Thank you
for your
time!
Questions?

}

..